

Code-Layout, Readability and Reusability

Date	01 July 2026
Team ID	SWTID-2026-2030
Project Name	Voice Based Concept Understanding Analyser
Maximum Marks	5 Marks

Code-Layout, Readability and Reusability:

The Voice-Based Concept Understanding Analyser is developed using a modular and well-structured codebase. The project follows proper coding standards, including meaningful variable and function names, consistent indentation, and clear comments for better readability. Functionalities such as speech-to-text conversion, concept analysis, audio visualization, PDF report generation, and the Streamlit user interface are implemented as separate reusable modules. This modular design improves maintainability, simplifies debugging, and allows the code to be extended for future AI-based voice analysis applications.

Code Layout Checklist:

S.No	Code Quality Parameter	Description	Followed (Yes / No / Partial)	Remarks
1	Consistent Indentation	Uniform spacing and indentation are maintained throughout the Python source code for better readability.	Yes	Four-space indentation is used consistently across all modules.
2	Proper File Structure	Files and folders are logically organized based on functionality.	Yes	The project is divided into modules such as app.py, speech_to_text.py, pdf_report.py, and utility files, making navigation easy.
3	Meaningful Variable Names	Variables are named according to their purpose and usage.	Yes	Variables like audio_file, transcript, concept_score, confidence_score, and uploaded_audio clearly represent their functionality.
4	Function /	Functions are named	Yes	Functions such as speech_to_text(),

	Method Names	descriptively to indicate their operations.		create_pdf(), analyze_concept(), and display_waveform() improve readability and maintainability.
5	Code Comments	Comments explain important sections and complex logic.	Yes	Inline comments and function descriptions are included to help understand the workflow and simplify future maintenance.
6	Modular Design	Code is divided into reusable and independent modules.	Yes	Speech recognition, concept analysis, audio visualization, report generation, and the user interface are implemented as separate modules.
7	No Redundant Code	Duplicate or unnecessary code has been removed.	Yes	Common functionality is implemented using reusable functions, reducing code duplication and improving maintainability.
8	Error Handling	Exceptions and runtime errors are handled appropriately.	Yes	The application validates audio input, handles speech recognition failures, and displays meaningful error messages to users instead of crashing.

Reusable Components / Modules:

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level
1	Speech-to-Text Module	Python + Whisper	Audio Processing	High
2	Audio Visualization	Python + Librosa + Matplotlib	Streamlit Interface	High
3	PDF Report Generator	Python + ReportLab	Report Generation	High
4	Concept Analysis Module	Sentence-BERT / NLP	Concept Evaluation	High
5	Streamlit User Interface	Streamlit	User Interaction	Medium

Overall Code Quality Assessment:

Aspect	Rating (1-5)	Comments
Code Layout & Structure	5	Well-organized project with modular architecture.
Readability	5	Clear naming conventions and proper formatting make the code easy to understand.
Reusability	5	Independent modules can be reused in future AI-based speech analysis projects.
Documentation / Comments	5	Important functions are documented with comments for easier maintenance.
Overall Score	5/5	Clean, modular, readable, and maintainable code following good software development practices.